SOFTWARE COMPANY
Palestine
+970 599 559 329

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 5

UNSUPERVISED LEARNING

Clustering, Dimensionality Reduction & Capstone Project Kickoff

The Phase 2→3 transition: learning from data that has no labels. Interns cluster data with K-Means and DBSCAN, reduce high-dimensional data with PCA and t-SNE, detect anomalies — then select their Phase 3 capstone project and run Sprint 1 planning.

PHASE 2 → 3 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

1. Week 5 Overview

Week	Week 5 of 10 — Phase 2 → Phase 3 transition
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 9-10 combined)
Format	On-site / Remote / Hybrid — all work in Jupyter notebooks committed to GitHub
Focus	Unsupervised learning: K-Means, DBSCAN, hierarchical clustering, PCA, t-SNE, anomaly detection; Phase 3 capstone project selection and Sprint 1 planning
Prerequisite	Weeks 1-4 completed: Python/EDA, supervised learning, evaluation, pipelines
Mentor Supervision	Daily check-in; Day 5 project selection and Sprint 1 plan require mentor sign-off before Phase 3 begins

2. Week 5 Learning Objectives

- Explain unsupervised learning and how it differs from supervised learning.
- Cluster data with K-Means and choose k using the elbow method and silhouette score.
- Cluster data with DBSCAN and hierarchical clustering, and explain when each is preferable.
- Reduce dimensionality with PCA and explain explained variance; visualize high-dimensional data with t-SNE.
- Detect anomalies in unlabeled data.
- Select a Phase 3 capstone project and complete Sprint 1 planning with a mentor-approved scope.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Unsupervised learning concepts; K-Means clustering and choosing k
Day 2	8 hrs	DBSCAN and hierarchical clustering; comparing clustering methods
Day 3	8 hrs	Dimensionality reduction: PCA and explained variance
Day 4	8 hrs	t-SNE for visualization; anomaly detection
Day 5	8 hrs	Phase 3 project selection; Sprint 1 planning and backlog

4. Day-by-Day Curriculum

Day 1 — Unsupervised Learning & K-Means — 8 hours

Learning Objectives

- Explain unsupervised learning and how it differs from supervised learning.
- Run K-Means clustering and interpret the resulting clusters and centroids.
- Choose the number of clusters k using the elbow method and silhouette score.

Key Topics

- Supervised vs. unsupervised learning
- What clustering does
- K-Means: the centroid-assignment loop
- Choosing k: the elbow method
- Choosing k: the silhouette score
- Scaling before clustering

Lesson Content

1.1 Supervised vs. Unsupervised Learning

In supervised learning (Weeks 3-4), every training example had a known correct answer (the target y). Unsupervised learning works on data with no labels at all — there is no y. Instead of predicting a known answer, the goal is to discover hidden structure the data contains on its own: natural groupings, underlying dimensions, or unusual points.

	Supervised	Unsupervised
Data	Has labels (X and y)	No labels (X only)
Goal	Predict the known target	Discover hidden structure
Examples	Regression, classification	Clustering, dimensionality reduction, anomaly detection
Evaluation	Compare prediction to true label	No ground truth — uses internal metrics and judgment

1.2 What Clustering Does

Clustering groups data points so that points in the same group are similar to each other and different from points in other groups. It answers questions like "what natural customer segments exist?" without anyone having pre-defined those segments. Because there are no labels, the algorithm finds structure purely from the shape of the data.

1.3 K-Means, Step by Step

K-Means is the most widely used clustering algorithm. It partitions data into a chosen number of clusters (k) by repeating two steps until stable:

- Step 1 — Place k cluster centers ("centroids"), initially at random.
- Step 2 — Assign each point to its nearest centroid.

- Step 3 — Move each centroid to the mean position of the points assigned to it.
- Repeat steps 2-3 until the centroids stop moving.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = km.fit_predict(X)
print(km.cluster_centers_)   # the final centroid positions
```

1.4 Choosing k: the Elbow Method

K-Means needs you to choose k in advance, but the right k is rarely obvious. The elbow method runs K-Means for a range of k values and plots the inertia (total within-cluster distance) against k. Inertia always falls as k rises, but the rate of improvement drops sharply at the right k — creating an "elbow" in the plot. That elbow is the recommended k.

```
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10).fit(X)
    inertias.append(km.inertia_)
plt.plot(range(1, 11), inertias, marker="o")   # look for the elbow
```

1.5 Choosing k: the Silhouette Score

The silhouette score is a more quantitative check: it measures how well each point sits inside its own cluster versus the nearest other cluster, on a scale from -1 to +1. A higher average silhouette score means better-defined clusters. Comparing the score across candidate k values gives a data-driven choice to confirm the elbow.

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, labels)
```

> *Always scale features before clustering. K-Means uses distance, so an unscaled large-range feature (like income) would dominate a small-range one (like age) — the same StandardScaler discipline from Week 4.*

Hands-On Lab: K-Means Clustering

- Step 1: Load and scale a provided dataset (numeric features only) with StandardScaler.
- Step 2: Run K-Means for k from 1 to 10 and plot inertia to find the elbow.
- Step 3: Compute the silhouette score for the top two candidate k values and pick the best.
- Step 4: Fit final K-Means with the chosen k, and visualize the clusters on a 2D scatter plot.
- Step 5: Interpret what each cluster represents in a Markdown cell.

Tools Used

Scikit-learn (KMeans) • StandardScaler • Matplotlib • Jupyter Notebook

Day 2 — DBSCAN & Hierarchical Clustering — 8 hours

Learning Objectives

- Explain K-Means's limitations and when to prefer another method.
- Run DBSCAN and interpret its clusters and noise points.
- Build and read a hierarchical clustering dendrogram, and choose a method per situation.

Key Topics

- Limitations of K-Means
- DBSCAN: density-based clustering and noise detection
- DBSCAN parameters: eps and min_samples

- Hierarchical clustering and dendrograms
- Choosing the right clustering method

Lesson Content

2.1 Why K-Means Isn't Always Enough

K-Means has real limitations: it requires you to pick k in advance, it assumes clusters are round and similarly sized, and it forces every point into a cluster — even clear outliers. When data has irregular shapes or noise, K-Means gives misleading results. Two alternative algorithms address these weaknesses.

2.2 DBSCAN

DBSCAN (Density-Based Spatial Clustering) groups points that are packed closely together and labels points in sparse regions as noise. It has two key advantages over K-Means: it discovers the number of clusters automatically (no k needed), and it can find arbitrarily shaped clusters while explicitly flagging outliers rather than forcing them into a group.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X)
# label -1 marks noise / outlier points
```

Parameter	Controls
eps	How close two points must be to count as neighbors
min_samples	How many neighbors a point needs to start a dense cluster

2.3 Hierarchical Clustering

Hierarchical clustering builds a tree of clusters — starting with every point as its own cluster and repeatedly merging the two closest, until all points join one cluster. The result is visualized as a dendrogram, a tree diagram where you can "cut" at any height to get any number of clusters. It needs no k in advance and shows the nested structure of the data.

```
from scipy.cluster.hierarchy import dendrogram, linkage
Z = linkage(X, method="ward")
dendrogram(Z)   # cut the tree at a chosen height to pick clusters
```

2.4 Choosing the Right Method

Method	Best When	Watch Out For
K-Means	Round, similarly-sized clusters; k roughly known	Forces outliers into clusters; needs k
DBSCAN	Irregular shapes, noise/outliers present	Sensitive to eps; struggles with varying densities
Hierarchical	You want nested structure or a dendrogram	Slow on very large datasets

Hands-On Lab: Comparing Clustering Methods

- Step 1: Run DBSCAN on the Day 1 dataset and report how many clusters and noise points it found.
- Step 2: Build a hierarchical clustering dendrogram and choose a cut height, noting the resulting cluster count.
- Step 3: Compare the K-Means, DBSCAN, and hierarchical results on the same data.
- Step 4: In Markdown, state which method best fits this dataset's shape and why.

Tools Used

Scikit-learn (DBSCAN) • SciPy (dendrogram) • Matplotlib • Jupyter Notebook

Day 3 — Dimensionality Reduction with PCA — 8 hours

Learning Objectives

- Explain the curse of dimensionality and why reduction helps.
- Apply PCA to reduce a dataset's dimensions.
- Interpret explained variance and choose how many components to keep.

Key Topics

- The curse of dimensionality
- What PCA does: principal components and variance
- Explained variance ratio
- Choosing the number of components (e.g. 95% variance)
- When (and when not) to use PCA

Lesson Content

3.1 The Curse of Dimensionality

Real datasets often have dozens or hundreds of features. High dimensionality causes real problems: data becomes sparse, distances lose meaning, models overfit more easily, and you cannot visualize beyond three dimensions. Dimensionality reduction compresses many features into a few, keeping as much information as possible while making data easier to model and visualize.

3.2 What PCA Does

Principal Component Analysis (PCA) finds new axes — "principal components" — that capture the directions of greatest variance in the data. The first component captures the most variance, the second the next most (at a right angle to the first), and so on. By keeping only the first few components, you reduce dimensions while retaining most of the information. Each component is a combination of the original features, built on the same linear-algebra operations from Week 2.

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)   # PCA requires scaling
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

3.3 Explained Variance

The key output is the explained variance ratio — how much of the data's total information each component keeps. Summing these tells you how much you retained after reduction. A common rule is to keep enough components to retain about 95% of the variance.

```
print(pca.explained_variance_ratio_)          # variance kept per component
print(pca.explained_variance_ratio_.sum())    # total variance retained
```

> *PCA requires scaled data, because it is variance-based: an unscaled high-range feature would appear artificially important. Always StandardScaler first.*

3.4 When to Use PCA

PCA has three main uses: compressing features to speed up and stabilize downstream models, reducing overfitting by removing redundant/correlated features, and reducing data to 2D or 3D so it can be plotted. The trade-off: the new components are combinations of features, so they lose the direct interpretability the original columns had.

Hands-On Lab: Reducing Dimensions with PCA

- Step 1: Scale a high-dimensional provided dataset with StandardScaler.
- Step 2: Fit PCA and plot the cumulative explained variance against the number of components.
- Step 3: Choose the number of components that retains ~95% of the variance and justify it.
- Step 4: Reduce the data to 2 components and produce a 2D scatter plot, coloring points by a known group if available.
- Step 5: Document what the reduction preserved and what it cost in Markdown.

Tools Used

Scikit-learn (PCA) • StandardScaler • Matplotlib • Jupyter Notebook

Day 4 — t-SNE & Anomaly Detection — 8 hours

Learning Objectives

- Use t-SNE to visualize high-dimensional data and distinguish it from PCA.
- Explain what anomaly detection is and why it is often unsupervised.
- Detect anomalies with Isolation Forest and interpret the flagged points.

Key Topics

- t-SNE for local-structure visualization
- PCA vs. t-SNE: when to use each
- What anomaly detection is
- Isolation Forest and the contamination parameter
- Anomaly detection and clustering overlap

Lesson Content

4.1 t-SNE for Visualization

t-SNE (t-distributed Stochastic Neighbor Embedding) is a dimensionality-reduction technique built specifically for visualization. Unlike PCA, which preserves global variance, t-SNE preserves local neighborhoods — it keeps points that were close together in high dimensions close together in 2D. This makes it excellent at revealing clusters visually, even when they aren't linearly separable.

```
from sklearn.manifold import TSNE
X_tsne = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X)
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=labels)
```

	PCA	t-SNE
Preserves	Global structure / variance	Local neighborhoods
Main use	Compression + visualization	Visualization only
Speed	Fast	Slow on large data
Axes meaning	Interpretable directions	No meaningful axes — only relative position

> *t-SNE is for looking, not for feeding into a model. Its axes are not meaningful and its output changes with settings — use it to see clusters, then use PCA (or original features) for actual modeling.*

4.2 What Anomaly Detection Is

Anomaly detection finds data points that differ significantly from the norm — fraud, defects, system failures, or errors. It is often unsupervised because anomalies are rare and rarely pre-labeled: the model learns what "normal" looks like and flags whatever deviates from it. This directly connects to the outline's fraud-detection capstone project option.

4.3 Isolation Forest

Isolation Forest is a standard anomaly-detection algorithm. Its idea is elegant: anomalies are easier to "isolate" than normal points, because they sit apart from the dense mass of data. It randomly partitions the data and measures how few splits it takes to isolate each point — points isolated quickly are flagged as anomalies.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05, random_state=42)
preds = iso.fit_predict(X)
# -1 = anomaly, 1 = normal
```

The contamination parameter is your estimate of the fraction of anomalies expected. DBSCAN's noise points (Day 2) are another simple form of anomaly detection, showing how these unsupervised techniques overlap.

Hands-On Lab: Visualization & Anomaly Detection

- Step 1: Apply t-SNE to reduce a high-dimensional dataset to 2D and plot it, coloring by cluster from Day 1-2.
- Step 2: Compare the t-SNE plot to the PCA plot from Day 3 and note what each reveals.
- Step 3: Run Isolation Forest on the dataset and report how many points were flagged as anomalies.
- Step 4: Inspect two flagged points and hypothesize why they were flagged, documented in Markdown.

Tools Used

Scikit-learn (TSNE, IsolationForest) • Matplotlib • Jupyter Notebook

Day 5 — Phase 3 Project Selection & Sprint 1 Planning — 8 hours

Learning Objectives

- Select a Phase 3 capstone project type in consultation with the mentor.
- Restate the project's Definition of Done and keep it visible from the start.
- Complete Sprint 1 planning: backlog, effort estimates, sprint goal, and acceptance criteria.

Key Topics

- Entering Phase 3: the four-sprint capstone
- The six capstone project options
- The professional baseline / Definition of Done
- Sprint 1 planning: backlog, effort, sprint goal
- Acceptance criteria and the GitHub branch/PR workflow

Lesson Content

5.1 Entering Phase 3

Phase 3 is the applied core of the program: each intern builds one complete AI/ML project end-to-end — from raw data to a deployed, usable product — across four one-week sprints (Weeks 6-9). Week 5 closes Phase 2 by selecting that project and planning its first sprint, so Week 6 begins with a clear, mentor-approved scope.

5.2 The Capstone Project Options

Interns choose one project type (or a comparable live BinX Tech use case), based on interest and current team needs:

Project	In Short
Customer Churn Prediction	Binary classification on telecom/SaaS data; EDA, feature importance, threshold tuning
House Price Prediction	Regression on real-estate data; pipelines, polynomial features, regularization
Sentiment Analysis Tool	NLP classifier for reviews/tweets (positive/neutral/negative); TF-IDF or a transformer
Image Classifier	CNN classifying images; data augmentation and transfer learning
Recommendation System	Collaborative or content-based filtering on interaction data
Fraud Detection Model	Imbalanced classification; SMOTE, precision-recall trade-offs, SHAP explanations

5.3 The Professional Baseline (Definition of Done)

Whatever the project type, every capstone must meet the same professional baseline, per the program's standard: a clean, documented Jupyter Notebook covering the full pipeline (EDA → preprocessing → modelling → evaluation); a trained model with reported metrics; a working deployment at a public URL (Streamlit or FastAPI); a GitHub repo with a clean README, requirements.txt, and model artifacts; and a short technical write-up. Keeping this end goal visible from Sprint 1 is what keeps the four sprints focused.

5.4 Sprint 1 Planning

Following the program's sprint structure, Sprint 1 planning means: selecting tasks from the ML project backlog (the natural first tasks are dataset selection, EDA, and a baseline model), estimating effort, and committing to a clear sprint goal. Sprint 1's goal is almost always "understand the data and establish a baseline model to beat" — the same baseline discipline emphasized since Week 3.

5.5 The Backlog and Acceptance Criteria

Each backlog task needs written acceptance criteria before work begins (per the program's per-task rules): the notebook cell runs without errors, code is committed to the correct feature branch with a clear message, a pull request is opened for mentor review before merging, results are documented in Markdown, and metrics are logged and compared to the baseline. Setting this structure up now is what makes Weeks 6-9 run smoothly.

Hands-On Lab: Project Kickoff & Sprint 1 Plan

- Step 1: Review the six project options with your mentor and select one based on your strengths and interests.
- Step 2: Write the project's problem statement and restate its Definition of Done.
- Step 3: Create the Sprint 1 backlog (dataset selection, EDA, baseline model) with written acceptance criteria per task.
- Step 4: Define the Sprint 1 goal and get mentor sign-off before any Phase 3 work begins.
- Step 5: Set up the project GitHub repository with a README skeleton and a feature-branch workflow.

Tools Used

Mentor consultation • GitHub (repository + branches) • Sprint backlog • Jupyter Notebook

5. Week 5 Deliverables

By the end of Week 5, every intern must submit the following to their mentor and GitHub repository:

- A K-Means notebook with elbow-method and silhouette analysis and interpreted clusters.
- A clustering-comparison notebook (K-Means vs. DBSCAN vs. hierarchical) with a method recommendation.
- A PCA notebook with a cumulative explained-variance plot and a justified component count.
- A t-SNE and anomaly-detection notebook with a 2D visualization and Isolation Forest results.
- A signed-off Phase 3 project selection, problem statement, and Sprint 1 plan with backlog and acceptance criteria.
- All Week 5 notebooks and the initialized project repository committed to GitHub.

6. Week 5 Evaluation Criteria

Scored by the assigned mentor at the end of Week 5 using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of unsupervised concepts	Runs algorithms but unclear on how they work	Explains clustering and reduction methods clearly	Deep understanding; justifies method choice per data shape
Clustering & reduction correctness	Basic use with guidance	Correct use, sensible k / component choices	Rigorous; validates with silhouette/variance, scales properly
Interpretation & insight	Reports output without meaning	Interprets clusters and components clearly	Insight-driven; connects structure to real meaning
Project planning quality	Vague scope or missing acceptance criteria	Clear project, backlog, and Sprint 1 goal	Well-scoped plan anticipating Phase 3 risks
Notebook quality & Git workflow	Sporadic commits, weak narrative	Regular commits, clear Markdown narrative	Consistent, descriptive, reproducible, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 5

Clustering	Scikit-learn (KMeans, DBSCAN), SciPy (hierarchical/dendrogram)
Dimensionality Reduction	Scikit-learn (PCA, TSNE)
Anomaly Detection	Scikit-learn (IsolationForest)
Evaluation	silhouette_score, explained_variance_ratio_
Environment	Python 3.10+, Pandas, Matplotlib, Jupyter Notebook, Git & GitHub

8. Notes & Best Practices

Unsupervised learning has no answer key, so judgment matters more than ever: scale before clustering, validate k with real metrics, and interpret results rather than trusting them blindly. Day 5 is the hinge into Phase 3 — no capstone work begins until the project scope and Sprint 1 plan are approved by the mentor, exactly as the program requires.